

Τι περιέχει το Μέρος 3

- 1 Ενημέρωση εγγράφων: `updateOne`, `updateMany`, `upsert`.
- 2 Διαγραφή: `deleteOne`, `deleteMany`.
- 3 Συνάθροιση: ροή σταδίων, στις συλλογές `country`, `books` και `persons`.
- 4 Αντιστοίχιση SQL και MongoDB, και η Εργασία 2 ενότητα προς ενότητα.

Οι ενημερώσεις γίνονται στη συλλογή `country` της Βραδιάς 1: Ελλάδα, Γαλλία, Γερμανία.

updateOne και \$set

Εντολές

```
use world
db.country.updateOne(
  { name: "Greece" },
  { $set: { capital: { name: "Athens", timezone: "Europe/Athens" } } }
)
```

Αποτελέσματα

```
switched to db world
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

- Πρώτο όρισμα: το φίλτρο.
Δεύτερο: η αλλαγή, με τελεστή.
- `$set` προσθέτει το πεδίο `capital`· αν υπήρχε, θα άλλαζε την τιμή του.
- Εργασία 2, ενότητα 11.

matchedCount και modifiedCount

Η ίδια εντολή, δεύτερη φορά

```
db.country.updateOne(
  { name: "Greece" },
  { $set: { capital: { name: "Athens", timezone: "Europe/Athens" } } }
)
```

Αποτέλεσμα

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
```

- Βρέθηκε ένα έγγραφο (`matchedCount: 1`), αλλά δεν άλλαξε (`modifiedCount: 0`): η τιμή ήταν ήδη αυτή.
- Με `matchedCount: 0`, το φίλτρο δεν βρήκε τίποτε: π.χ. "greece" με πεζό.

\$inc και \$push

Εντολές

```
db.country.updateOne(
  { name: "Greece" },
  { $inc: { visits: 1 }, $push: { languages: "Greek" } }
)
db.country.findOne({ name: "Greece" }, { _id: 0 })
```

Αποτελέσματα

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
{
  name: 'Greece',
  euSince: 1981,
  population: 10400000,
  capital: { name: 'Athens', timezone: 'Europe/Athens' },
  languages: [ 'Greek' ],
  visits: 1
}
```

- **\$inc**: αυξάνει έναν αριθμό· αν το πεδίο λείπει, το δημιουργεί.
- **\$push**: προσθέτει στοιχείο σε λίστα· αν η λίστα λείπει, τη δημιουργεί.
- Δύο τελεστές στην ίδια ενημέρωση.

\$unset: αφαίρεση πεδίου

Εντολή

```
db.country.updateOne({ name: "Greece" }, { $unset: { visits: "" } })
```

Αποτέλεσμα

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

- Το πεδίο visits αφαιρείται· η τιμή μετά το όνομα δεν έχει σημασία, συνήθως "".
- Με updateMany, το ίδιο πεδίο αφαιρείται από όλα τα έγγραφα που ταιριάζουν (διαφάνεια 8).

Οι τελεστές ενημέρωσης

Τελεστής	Τι κάνει
<code>\$set</code>	ορίζει την τιμή ενός πεδίου· το δημιουργεί αν λείπει
<code>\$unset</code>	αφαιρεί ένα πεδίο
<code>\$inc</code>	αυξάνει έναν αριθμό κατά μια τιμή
<code>\$rename</code>	μετονομάζει ένα πεδίο
<code>\$push</code>	προσθέτει στοιχείο σε λίστα
<code>\$addToSet</code>	προσθέτει στοιχείο σε λίστα, μόνο αν δεν υπάρχει ήδη
<code>\$pull</code>	αφαιρεί από λίστα όσα στοιχεία ταιριάζουν σε μια συνθήκη

Πολλοί τελεστές στην ίδια ενημέρωση: `{ $set: { ... }, $inc: { ... } }`.



Προσοχή

updateOne: μόνο το πρώτο έγγραφο

Εντολές

```
db.country.updateOne({ euSince: 1958 }, { $set: { founder: true } })
db.country.countDocuments({ founder: true })
db.country.updateMany({ euSince: 1958 }, { $set: { founder: true } })
```

Αποτελέσματα

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
1
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 1,
  upsertedCount: 0
}
```

- Γαλλία και Γερμανία έχουν euSince 1958· το updateOne άλλαξε μόνο την πρώτη.
- Το updateMany βρήκε και τις δύο (matchedCount: 2) και άλλαξε όποια δεν είχε ήδη την τιμή (modifiedCount: 1).

updateMany: από «0 σελίδες» σε «άγνωστο»

Εντολές

```
use library
db.books.updateMany({ pageCount: 0 }, { $unset: { pageCount: "" } })
db.books.countDocuments({ pageCount: { $exists: false } })
```

Αποτελέσματα

```
switched to db library
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 166,
  modifiedCount: 166,
  upsertedCount: 0
}
166
```

- Ένα φίλτρο, πολλά έγγραφα: και τα 166 βιβλία με 0 σελίδες.
- Τώρα το «άγνωστο» είναι πεδίο που λείπει, όχι ψεύτικο 0: δεν μετρά στους μέσους όρους.
- Επαναφορά του αρχείου:
`mongoimport ... --drop`.

upsert: η Κύπρος επιστρέφει

Εντολές

```
use world
db.country.updateOne(
  { name: "Cyprus" },
  { $set: { population: 980000 } },
  { upsert: true }
)
db.country.findOne({ name: "Cyprus" }, { _id: 0 })
```

Αποτελέσματα

```
switched to db world
{
  acknowledged: true,
  insertedId: ObjectId('6ab42c38413cb46e99da303c'),
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 1
}
{ name: 'Cyprus', population: 980000 }
```

- Κανένα έγγραφο δεν ταίριαξε: με `upsert: true` δημιουργείται νέο.
- Το νέο έγγραφο παίρνει το name από το φίλτρο και το population από το `$set`.

🟡 Ερώτημα

Τι κάνει αυτή η εντολή;

Η Ελλάδα έχει ήδη πέντε πεδία.

```
db.country.updateOne({ name: "Greece" },  
                      { population: 10500000 })
```

- A** Αλλάζει μόνο τον πληθυσμό
- B** Αντικαθιστά όλο το έγγραφο
- Γ** Εμφανίζει σφάλμα
- Δ** Δεν κάνει τίποτε, χωρίς μήνυμα

01:00

Η απάντηση: σφάλμα

Εντολή

```
db.country.updateOne({ name: "Greece" },  
                     { population: 10500000 })
```

Αποτέλεσμα

```
MongoInvalidArgumentError: Update document requires atomic operators
```

- Το `updateOne` θέλει τελεστή: `$set`, `$inc`, `$unset` ...
- Για αντικατάσταση ολόκληρου του εγγράφου υπάρχει το `replaceOne`.
- Προσοχή σε παλιούς οδηγούς: το `update()` χωρίς τελεστή αντικαθιστούσε όλο το έγγραφο.

Πριν από κάθε μαζική διαγραφή: μέτρηση

Εντολή

```
db.country.countDocuments({ population: { $lt: 1000000 } })
```

Αποτέλεσμα

1

- Πρώτα `countDocuments` με το φίλτρο της διαγραφής: 1, η Κύπρος.
- `deleteMany({})` και `updateMany({}, ...)` αφορούν όλη τη συλλογή, χωρίς επιβεβαίωση και χωρίς αναίρεση.

deleteMany και deleteOne

Εντολή

```
db.country.deleteMany({ population: { $lt: 1000000 } })
```

Αποτέλεσμα

```
{ acknowledged: true, deletedCount: 1 }
```

- `deleteMany`: όλα όσα ταιριάζουν· εδώ ένα, η Κύπρος.
- `deleteOne`: μόνο το πρώτο, όπως στη Βραδιά 1.
- Εργασία 2, ενότητα 12.

Συνάθροιση: ροή σταδίων

\$match

Φιλτράρει έγγραφα, όπως το φίλτρο του find.

→

\$group

Ομαδοποιεί και υπολογίζει ανά ομάδα: πλήθος, άθροισμα, μέσο όρο, ελάχιστο, μέγιστο.

→

\$sort, \$limit, \$project

Ταξινομούν, περιορίζουν και διαμορφώνουν την έξοδο.

Σύνταξη

```
db.συλλογή.aggregate([  
  { στάδιο 1 },  
  { στάδιο 2 },  
  ...  
])
```

Η έξοδος κάθε σταδίου είναι η είσοδος του επόμενου.

\$group: ο συνολικός πληθυσμός

Εντολή

```
db.country.aggregate([
  { $group: { _id: null, totalPopulation: { $sum: "$population" } } }
])
```

Αποτέλεσμα

```
[ { _id: null, totalPopulation: 162600000 } ]
```

- `_id: null`: μία μόνο ομάδα, όλα τα έγγραφα μαζί.
- `"$population"`: η τιμή του πεδίου· το \$ μπροστά από το όνομα.
- Ελλάδα, Γαλλία και Γερμανία: 162.600.000.

\$group ανά πεδίο: βιβλία ανά κατάσταση

Εντολές

```
use library
db.books.aggregate([
  { $group: { _id: "$status", books: { $sum: 1 } } }
])
```

Αποτελέσματα

```
switched to db library
[ { _id: 'PUBLISH', books: 363 }, { _id: 'MEAP', books: 68 } ]
```

- `_id: "$status"`: μία ομάδα για κάθε τιμή του status.
- `$sum: 1`: μετρά τα έγγραφα κάθε ομάδας, όπως το COUNT(*) με GROUP BY.
- Εργασία 2, ενότητα 10.

\$match και συσσωρευτές

Εντολή

```
db.books.aggregate([
  { $match: { categories: "Java" } },
  { $group: { _id: "$status", books: { $sum: 1 },
    avgPages: { $avg: "$pageCount" },
    maxPages: { $max: "$pageCount" } } }
])
```

Αποτέλεσμα

```
[
  { _id: 'MEAP', books: 1, avgPages: 700, maxPages: 700 },
  {
    _id: 'PUBLISH',
    books: 95,
    avgPages: 511.4868421052632,
    maxPages: 1088
  }
]
```

- `$match` πρώτο: μόνο τα βιβλία Java, όπως το WHERE.
- `$avg` και `$max`: μέσος όρος και μέγιστο ανά ομάδα.
- Το `$avg` αγνοεί τα βιβλία που δεν έχουν πλέον `pageCount` (διαφάνεια 8).

\$unwind: ένα έγγραφο ανά κατηγορία

Εντολή

```
db.books.aggregate([
  { $unwind: "$categories" },
  { $group: { _id: "$categories", books: { $sum: 1 } } },
  { $sort: { books: -1 } },
  { $limit: 5 }
])
```

Αποτέλεσμα

```
[
  { _id: 'Java', books: 96 },
  { _id: 'Internet', books: 41 },
  { _id: 'Microsoft .NET', books: 34 },
  { _id: 'Web Development', books: 17 },
  { _id: 'Software Engineering', books: 16 }
]
```

- `$unwind`: ένα έγγραφο για κάθε κατηγορία κάθε βιβλίου.
- Μετά: ομάδα ανά κατηγορία, ταξινόμηση, τα πέντε πρώτα.
- Βιβλία χωρίς κατηγορία δεν περνούν από το `$unwind`.

\$match μετά το \$group: το HAVING

Εντολή

```
db.books.aggregate([
  { $unwind: "$categories" },
  { $group: { _id: "$categories", books: { $sum: 1 } } },
  { $match: { books: { $gte: 12 } } },
  { $sort: { books: -1, _id: 1 } }
])
```

Αποτέλεσμα

```
[
  { _id: 'Java', books: 96 },
  { _id: 'Internet', books: 41 },
  { _id: 'Microsoft .NET', books: 34 },
  { _id: 'Web Development', books: 17 },
  { _id: 'Software Engineering', books: 16 },
  { _id: 'Business', books: 12 },
  { _id: 'Programming', books: 12 }
]
```

- `$match` μετά το `$group` φιλτράρει ομάδες, όπως το HAVING της SQL.
- Κατηγορίες με τουλάχιστον 12 βιβλία: επτά.
- `_id: 1` στο `$sort`: σταθερή σειρά στις ισοπαλίες.

\$project: το σχήμα της εξόδου

Εντολή

```
db.books.aggregate([
  { $match: { categories: "Java" } },
  { $group: { _id: "$status", books: { $sum: 1 },
    avgPages: { $avg: "$pageCount" } } },
  { $project: { _id: 0, status: "$_id", books: 1,
    avgPages: { $round: ["$avgPages", 0] } } }
])
```

Αποτέλεσμα

```
[
  { books: 1, status: 'MEAP', avgPages: 700 },
  { books: 95, status: 'PUBLISH', avgPages: 511 }
]
```

- `$project`: ποια πεδία βγαίνουν και με ποιο όνομα· το `_id` γίνεται status.
- `$round`: στρογγυλοποίηση του μέσου όρου.
- Ό,τι είναι το SELECT για την SQL.

To persons.txt στο mongosh

Το αρχείο persons.txt

```
db = db.getSiblingDB("book-filtered-top-subset");
db.dropDatabase();
db.persons.createIndex({"vocation": 1, "dateofbirth": 1});
db.persons.insertMany([ ... έξι άτομα ... ]);
```

Αποτέλεσμα της τελευταίας εντολής

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ab42c36337843d01ed17eaf'),
    '1': ObjectId('6ab42c36337843d01ed17eb0'),
    '2': ObjectId('6ab42c36337843d01ed17eb1'),
    '3': ObjectId('6ab42c36337843d01ed17eb2'),
    '4': ObjectId('6ab42c36337843d01ed17eb3'),
    '5': ObjectId('6ab42c36337843d01ed17eb4')
  }
}
```

- Ανοίξτε το αρχείο, αντιγράψτε όλο το περιεχόμενο και επικολλήστε το στο mongosh.
- Νέα βάση με έξι άτομα· η διεύθυνση είναι εμφωλευμένο έγγραφο.
- Εναλλακτικά:
`load("persons.txt")`, με το mongosh ανοιγμένο στον φάκελο του αρχείου.

Οι τρεις νεότεροι μηχανικοί

Εντολές

```
var pipeline = [
  { $match: { vocation: "ENGINEER" } },
  { $sort: { dateofbirth: -1 } },
  { $limit: 3 },
  { $unset: [ "_id", "vocation", "address" ] }
]
db.persons.aggregate(pipeline)
```

Αποτέλεσμα

```
[
  {
    person_id: '7363626383',
    firstname: 'Carl',
    lastname: 'Simmons',
    dateofbirth: ISODate('1998-12-26T13:13:55.000Z')
  },
  {
    person_id: '1723338115',
    firstname: 'Olive',
    lastname: 'Ranieri',
    dateofbirth: ISODate('1985-05-12T23:14:30.000Z'),
    gender: 'FEMALE'
  },
  {
    person_id: '6392529400',
    firstname: 'Elise',
    lastname: 'Smith',
    dateofbirth: ISODate('1972-01-13T09:32:07.000Z')
  }
]
```

Φιλτράρει τους μηχανικούς, ταξινομεί από τον νεότερο, κρατά τρεις και αφαιρεί τρία πεδία.

\$unset ως στάδιο

```
// με $unset
{ $unset: [ "_id", "vocation", "address" ] }

// με $project
{ $project: {
  _id: 0, vocation: 0, address: 0 } }
```

- Και τα δύο δίνουν τους ίδιους τρεις μηχανικούς.
- `$unset`: γράφει μόνο όσα αφαιρούνται.
- `$project`: και για προσθήκη ή μετονομασία πεδίων (διαφάνεια 20).

Το ίδιο με find

Εντολή

```
db.persons.find(
  { vocation: "ENGINEER" },
  { _id: 0, vocation: 0, address: 0 }
).sort({ dateofbirth: -1 }).limit(3)
```

Αποτέλεσμα

```
[
  {
    person_id: '7363626383',
    firstname: 'Carl',
    lastname: 'Simmons',
    dateofbirth: ISODate('1998-12-26T13:13:55.000Z')
  },
  {
    person_id: '1723338115',
    firstname: 'Olive',
    lastname: 'Ranieri',
    dateofbirth: ISODate('1985-05-12T23:14:30.000Z'),
    gender: 'FEMALE'
  },
  {
    person_id: '6392529400',
    firstname: 'Elise',
    lastname: 'Smith',
    dateofbirth: ISODate('1972-01-13T09:32:07.000Z')
  }
]
```

Χωρίς ομαδοποίηση, το find αρκεί· η συνάθροιση χρειάζεται όταν υπολογίζουμε ανά ομάδα.

Η ενότητα 10 ζητά ομαδοποίηση

Ερώτημα με pipeline που παρουσιάζει συγκεντρωτικά, ομαδοποιημένα αποτελέσματα.

Εργασία 2, ενότητα 10

Το παράδειγμα του persons

\$match, \$sort, \$limit, \$unset: φιλτράρει και ταξινομεί, αλλά δεν ομαδοποιεί.

Για την Εργασία 2

Τουλάχιστον ένα \$group, με τα δικά σας δεδομένα, όπως στις διαφάνειες 15 έως 20.

Ομαδοποίηση στο persons: άτομα ανά επάγγελμα

Εντολή

```
db.persons.aggregate([
  { $group: { _id: "$vocation", people: { $sum: 1 },
    oldest: { $min: "$dateofbirth" } } },
  { $sort: { people: -1, _id: 1 } }
])
```

Αποτέλεσμα

```
[
  {
    _id: 'ENGINEER',
    people: 4,
    oldest: ISODate('1959-07-06T17:35:45.000Z')
  },
  {
    _id: 'FLORIST',
    people: 1,
    oldest: ISODate('1941-04-07T22:11:52.000Z')
  },
  {
    _id: 'POLITICIAN',
    people: 1,
    oldest: ISODate('1991-11-23T16:53:56.000Z')
  }
]
```

Μία ομάδα ανά επάγγελμα: πόσα άτομα, και η παλαιότερη ημερομηνία γέννησης (`$min`).

SQL και MongoDB: εντολές

SQL	MongoDB
INSERT INTO ... VALUES ...	insertOne, insertMany
SELECT ... FROM ... WHERE ...	find(φίλτρο, προβολή)
UPDATE ... SET ... WHERE ...	updateOne, updateMany με \$set
SET age = age + 3	\$inc: { age: 3 }
DELETE FROM ... WHERE ...	deleteOne, deleteMany με φίλτρο
DELETE FROM people	deleteMany({})
ALTER TABLE ... ADD ...	updateMany({}, { \$set: ... })
ALTER TABLE ... DROP COLUMN ...	updateMany({}, { \$unset: ... })

Στη MongoDB δεν υπάρχει ALTER TABLE: η συλλογή δεν επιβάλλει δομή, και τα πεδία αλλάζουν στο επίπεδο του εγγράφου.

SQL και MongoDB: συνάθροιση

SQL	Συνάθροιση στη MongoDB
WHERE	<code>\$match</code>
GROUP BY	<code>\$group</code>
HAVING	<code>\$match</code> μετά το <code>\$group</code>
SELECT	<code>\$project</code>
ORDER BY	<code>\$sort</code>
LIMIT	<code>\$limit</code>
<code>SUM()</code> , <code>COUNT()</code>	<code>\$sum</code> , με <code>\$sum: 1</code> για το πλήθος
JOIN	<code>\$lookup</code>

Το `$lookup` συνδέει δύο συλλογές, όπως το JOIN. Η Εργασία 2 δεν το ζητά.

Εργασία 2: κάθε ενότητα και πού διδάχθηκε

Ενότητα της εκφώνησης	Πού διδάχθηκε
1–3. Βάση, συλλογή, έγγραφα	Βραδιά 1, Μέρος 3, διαφάνειες 12–16
4. Προβολή πεδίων	Μέρος 1, διαφάνειες 9–10
5. Λογικοί τελεστές	Μέρος 1, διαφάνειες 17–19
6. Τελεστές σύγκρισης	Μέρος 1, διαφάνειες 13–14
7. Regex	Μέρος 1, διαφάνειες 24–25· Άσκηση 2
8. Σύνθετα ερωτήματα	Μέρος 1, διαφάνεια 18 και Άσκηση 1· φύλλο του Μέρους 2
9. Ταξινόμηση και περιορισμός	Μέρος 1, διαφάνειες 29–31
10. Pipeline με ομαδοποίηση	Μέρος 3, διαφάνειες 15–20 και 26
11. Ενημέρωση	Μέρος 3, διαφάνειες 2–9
12. Διαγραφή	Βραδιά 1, Μέρος 3, διαφάνεια 22· Μέρος 3, διαφάνεια 13
JSON και μεταβλητές	Μέρος 1, διαφάνειες 11–12

Εργασία 2: τι παραδίδετε

Αναφορά

Ονοματεπώνυμο, email, τίτλος· περιγραφή των τροποποιήσεων της βάσης της Εργασίας 1 και των πινάκων (τουλάχιστον 2, ο ένας εμφωλευμένος στον άλλον).

Ενότητες 1 έως 12

Αριθμημένες όπως στην εκφώνηση· για κάθε κώδικα, μία παράγραφος που εξηγεί τι κάνει και στιγμιότυπο οθόνης με το αποτέλεσμα.

JSON και μεταβλητές

Σε κάποια ερωτήματα, ευανάγνωστο JSON και μεταβλητές, με κείμενο που το επισημαίνει.

Προθεσμία

Στην πλατφόρμα του μαθήματος.

Τρία σημεία από το Μέρος 3

- 1 Κάθε ενημέρωση έχει φίλτρο και τελεστή: το `updateOne` αλλάζει το πρώτο έγγραφο, το `updateMany` όλα, και χωρίς τελεστή εμφανίζεται σφάλμα.
- 2 Η συνάθροιση είναι ροή σταδίων: το `$match` φιλτράρει, το `$group` ομαδοποιεί, τα `$sort`, `$limit` και `$project` διαμορφώνουν την έξοδο.
- 3 Πριν από κάθε μαζική αλλαγή ή διαγραφή, `countDocuments` με το ίδιο φίλτρο.

Ερωτήσεις

Εδώ κλείνουν οι μη σχεσιακές βάσεις δεδομένων. Στο επόμενο μάθημα ξεκινά η Μαθησιακή Αναλυτική.